

MWORKS专项培训系列（五）

Syslab控制工具箱基础使用方法

王春雨

苏州同元软控信息技术有限公司

2025年5月15日



@苏州同元软控信息技术有限公司版权所有，侵权必究
未经许可，不得复印、传播或以其他方式使用

目录

1. 控制系统的数学模型

2. 控制系统的时域分析

3. 控制系统的频域分析

1. 控制系统的数学模型-模型的创建

大部分控制系统分析与设计的方法都需要假设系统的模型已知，而获得数学模型有两种方法：

- 从已知的物理规律出发，用数学推导的方式建立系统的数学模型
- 由实验数据拟合系统的数学模型（系统辨识）



线性系统一般可以用以下两种模型进行表示：

1. 传递函数模型
2. 状态空间模型

传递函数模型 (tf)

连续时间动态系统一般以微分方程描述，而 LTI 系统则以定常系数线性常微分方程描述。假设系统的输入信号为 $u(t)$ ，输出为 $y(t)$ ，则 n 阶系统的微分方程为：

$$a_n \frac{d^n y(t)}{dt^n} + a_{n-1} \frac{d^{n-1} y(t)}{dt^{n-1}} + \dots + a_1 \frac{dy(t)}{dt} + a_0 y(t) = b_m \frac{d^m u(t)}{dt^m} + b_{m-1} \frac{d^{m-1} u(t)}{dt^{m-1}} + \dots + b_1 \frac{du(t)}{dt} + b_0 u(t) \quad (n \geq m)$$

系统输入量与输出量的 Laplace 变换之比即为其传递函数：

$$G(s) = \frac{Y(s)}{U(s)} = \frac{b_m s^m + b_{m-1} s^{m-1} + \dots + b_1 s + b_0}{a_n s^n + a_{n-1} s^{n-1} + \dots + a_1 s + a_0}, n \geq m$$

传递函数可以表示成两个多项式的比值，在 Syslab 中，多项式可以用向量表示。将多项式的系数按 s 的降幂次序表示得到一个数值向量，分别用 num 与 den 表示分子、分母多项式，再利用 `tf()` 函数即可创建系统传递函数

```
num = [bm, ... b1, b0]
den = [an, ... a1, a0]
# 通过指定分子分母多项式系数创建传递函数
G = tf(num, den)
```

1. 控制系统的数学模型-模型的创建

传递函数模型 (tf)

实操1: Syslab中建立传递函数模型:

$$G(s) = \frac{12s^3 + 24s^2 + 12s + 20}{2s^4 + 4s^3 + 6s^2 + 2s + 2}$$

方式一

```
num = [12,24,12,20]
```

```
den = [2,4,6,2,2]
```

```
G = tf(num,den)
```

方式二

```
G = tf([12,24,12,20],[2,4,6,2,2])
```

输出:

$$12s^3 + 24s^2 + 12s + 20$$
$$2s^4 + 4s^3 + 6s^2 + 2s + 2$$

连续时间传递函数模型

考虑如果传递函数分子分母多项式给出的不是完全展开的形式, 而是若干因式的乘积, 或者包括其他运算。这种情况可以定义Laplace算子: `s = tf('s')`, 然后用类似数学表达式的形式直接输入

实操2: Syslab中建立传递函数模型:

$$G(s) = \frac{3(s^2 + 3)}{(s+2)^3(s^2 + 2s + 1)(s^2 + 5)}$$

```
s = tf('s') # 定义Laplace算子
```

```
G = 3*(s^2+3)/((s+2)^3*(s^2+2*s+1)*(s^2+5))
```

输出:

$$3s^2 + 9$$
$$s^7 + 8s^6 + 30s^5 + 78s^4 + 153s^3 + 198s^2 + 140s + 40$$

连续时间传递函数模型

1. 控制系统的数学模型-模型的创建

传递函数模型 (tf)

离散时间动态系统一般以差分方程描述，LTI 系统则以定系数线性差分方程描述，对于离散SISO，设定采样周期为 T ，系统的输入为 $u(i)$ ，输出为 $y(i)$ ，则相应差分方程为：

$$a_n y(i+n) + a_{n-1} y(i+n-1) + \cdots + a_1 y(i+1) + a_0 y(i) = b_m u(i+m) + b_{m-1} u(i+m-1) + \cdots + b_1 u(i+1) + b_0 u(i)$$

对上述方程进行 z 变换，得到离散系统的传递函数：

$$H(z) = \frac{Y(z)}{U(z)} = \frac{b_m z^m + b_{m-1} z^{m-1} + \cdots + b_1 z + b_0}{a_n z^n + a_{n-1} z^{n-1} + \cdots + a_1 z + a_0}, n \geq m$$

在 Syslab 中，同样使用 `tf()` 函数创建离散系统传递函数，与连续传递函数不同的是，同时需要指定采样时间 T

示例3： Syslab中建立离散系统传递函数模型，其采样周期为 $T = 0.1s$

$$H(z) = \frac{6z^2 - 0.6z - 0.12}{z^4 - z^3 + 0.25z^2 + 0.25z - 0.125}$$

```
num = [bm, ... b1,b0]
```

```
den = [an, ... a1,a0]
```

```
# 通过指定分子分母多项式系数、采样时间
```

```
G = tf(num,den,ts)
```

```
num = [6 -0.6 -0.12]
```

```
den = [1 -1 0.25 0.25 -0.125]
```

```
H = tf(num,den,0.1)
```

输出：

$$6.0z^2 - 0.6z - 0.12$$
$$1.0z^4 - 1.0z^3 + 0.25z^2 + 0.25z - 0.125$$

Sample Time: 0.1 (seconds)

离散时间传递函数模型

Tips:

针对离散时间系统模型，同样可以通过 $z = \text{tf}('z',Ts)$ 定义 z 算子，再进行数学表达式的形式创建模型

1. 控制系统的数学模型-模型的创建

零极点增益模型 (zpk)

零极点增益 (Zero-Pole-Gain) 模型是传递函数模型的一种特殊形式，其将系统表示为零点 (Zeros)、极点 (Poles) 和增益 (Gain) 相乘的形式：

$$G(s) = k \frac{\prod_{i=1}^m (s + z_i)}{\prod_{j=1}^n (s + p_j)} = k \frac{(s + z_1)(s + z_2) \cdots (s + z_m)}{(s + p_1)(s + p_2) \cdots (s + p_n)}$$

其中：k 为系统增益， $-z_i, i = 1, 2, \dots, m$ 为系统零点， $-p_j, j = 1, 2, \dots, n$ 为系统极点

实操4：构建下面的零极点增益模型

$$G(s) = \frac{-2s}{(s-1-i)(s-1+i)(s-2)}$$

```
z = [0]
p = [1-1im, 1+1im, 2]
k = -2
G = zpk(z, p, k)
```

输出

$$\frac{-2s}{(s^2 - 2s + 2)(s - 2)}$$

连续时间传递函数模型

示例5：构建离散时间零极点增益模型，采样时间为 $ts = 0.125s$

$$H(z) = \frac{\frac{z}{z-0.3} \cdot 2(z+0.5)}{(z-0.1+i)(z-0.1-i)}$$

```
z = (0, -0.5)
p = (0.3, [0.1-1im, 0.1+1im])
k = [1, 2]
G = zpk(z, p, k, 0.125)
```

输出

输入 1 到输出 1

$$\frac{1.0z}{1.0z - 0.3}$$

输入 1 到输出 2

$$\frac{1.0z + 0.5}{1.0z^2 - 0.2z + 1.01}$$

Sample Time: 0.125 (seconds)
离散时间传递函数模型

1. 控制系统的数学模型-模型的创建

MIMO传递函数模型 (tf)

多变量系统模型的一种表述形式为：传递函数矩阵。这是单变量系统传递函数在多变量系统中的直接扩展，一般可写为：

$$G(s) = \begin{bmatrix} G_{11}(s) & G_{12}(s) & \cdots & G_{1n}(s) \\ G_{21}(s) & G_{22}(s) & \cdots & G_{2n}(s) \\ \vdots & \vdots & \ddots & \vdots \\ G_{m1}(s) & G_{m2}(s) & \cdots & G_{mn}(s) \end{bmatrix}$$

其中 $G_{i,j}(s)$ 表示第 i 路输入信号对第 j 路输出信号的放大倍数

- 方式一：通过创建的子传递函数构造MIMO系统传递函数矩阵
- 方式二：通过元组来指定分子和分母的系数构造MIMO系统传递函数矩阵

示例6：构建以下MIMO系统的传递函数矩阵

$$G(s) = \begin{bmatrix} \frac{s-1}{s+1} & \frac{100}{(s+4)(s+10.625)} \\ \frac{s+2}{s^2+4s+5} & \frac{s+2}{s^3+4s+20} \end{bmatrix}$$

```
s = tf('s')
G11 = tf([1,-1],[1,1])
G12 = 100/((s+4)*(s+10.625))
G21 = tf([1,2],[1,4,5])
G22 = tf([1,2],[1,0,4,20])
# 通过矩阵元构造传递函数矩阵
G = [G11 G12;G21 G22]
```

输出

输入 1 到输出 1
 $\frac{1.0s - 1.0}{1.0s + 1.0}$
 $1.0s + 1.0$

输入 1 到输出 2
 $\frac{1.0s + 2.0}{1.0s^2 + 4.0s + 5.0}$
 $1.0s^2 + 4.0s + 5.0$

输入 2 到输出 1
 $\frac{100.0}{1.0s^2 + 14.625s + 42.5}$
 $1.0s^2 + 14.625s + 42.5$

输入 2 到输出 2
 $\frac{1.0s + 2.0}{1.0s^3 + 4.0s + 20.0}$
 $1.0s^3 + 4.0s + 20.0$

连续时间传递函数模型

示例7：构建以下离散MIMO系统的传递函数矩阵，采样时间为 $ts = 0.2s$

$$H(z) = \begin{bmatrix} \frac{1}{z+0.3} & \frac{z}{z+0.3} \\ \frac{-z+2}{z+0.3} & \frac{3}{z+0.3} \end{bmatrix}$$

```
nums = ([1],[-1,2],[1,0],[3])
dens = ([1,0.3],[1,0.3],[1,0.3],[1,0.3])
ts = 0.2
H = tf(nums,dens,2,2,ts)
#tf(nums,dens,ny,nu,ts)
#ny 和 nu 分别为输出和输入的通道数
```

输出

输入 1 到输出 1
 $\frac{1.0}{1.0z + 0.3}$
 $1.0z + 0.3$

输入 1 到输出 2
 $\frac{-1.0z + 2.0}{1.0z + 0.3}$
 $1.0z + 0.3$

输入 2 到输出 1
 $\frac{1.0z}{1.0z + 0.3}$
 $1.0z + 0.3$

输入 2 到输出 2
 $\frac{3.0}{1.0z + 0.3}$
 $1.0z + 0.3$

Sample Time: 0.2 (seconds)
离散时间传递函数模型



1. 控制系统的数学模型-模型的创建

二. 状态空间模型 (ss)

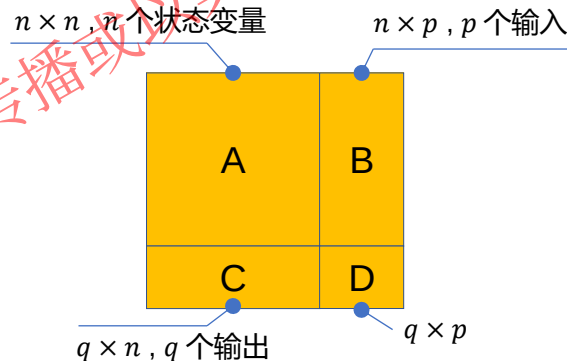
状态空间模型可以描述更广的一类控制系统，包括非线性系统、MIMO系统。针对连续时间 LTI 系统可以描述为：

□ 针对**连续时间** LTI 系统可以描述为：

$$\begin{cases} \dot{x}(t) = Ax(t) + Bu(t) \\ y(t) = Cx(t) + Du(t) \end{cases}$$

□ 针对**离散时间** LTI 系统可以描述为：

$$\begin{cases} x[n+1] = Ax[n] + Bu[n] \\ y[n] = Cx[n] + Du[n] \end{cases}$$



实操8：构建下面的双输入双输出系统状态空间模型

$$\begin{cases} \dot{x}(t) = \begin{bmatrix} -12 & -17.2 & -16.8 & -11.9 \\ 6 & 8.5 & 9.5 & 8 \\ 5 & 8.7 & 3.5 & 6 \\ -6 & -6.5 & -9.7 & -5 \end{bmatrix} x(t) + \begin{bmatrix} 2.5 & 0.1 \\ 2 & 0.5 \\ 3 & 1 \\ 0 & 0.5 \end{bmatrix} u(t) \\ y(t) = \begin{bmatrix} 2 & 3 & 0.8 & 0 \\ 0.5 & 0.56 & 0.3 & -1 \end{bmatrix} x(t) \end{cases}$$

```
A = [-12 -17.2 -16.8 -11.9; 6 8.5 9.5 8; 5 8.7 3.5 6; -6 -6.5 -9.7 -5]
B = [2.5 0.1; 2 0.5; 3 1; 0 0.5]
C = [2 3 0.8 0; 0.5 0.56 0.3 -1]
D = zeros(2,2)
G = ss(A,B,C,D)
```

输出

```
A =
-12.0 -17.2 -16.8 -11.9
  6.0  8.5  9.5  8.0
  5.0  8.7  3.5  6.0
 -6.0 -6.5 -9.7 -5.0

B =
  2.5  0.1
  2.0  0.5
  3.0  1.0
  0.0  0.5

C =
  2.0  3.0  0.8  0.0
  0.5  0.56 0.3 -1.0

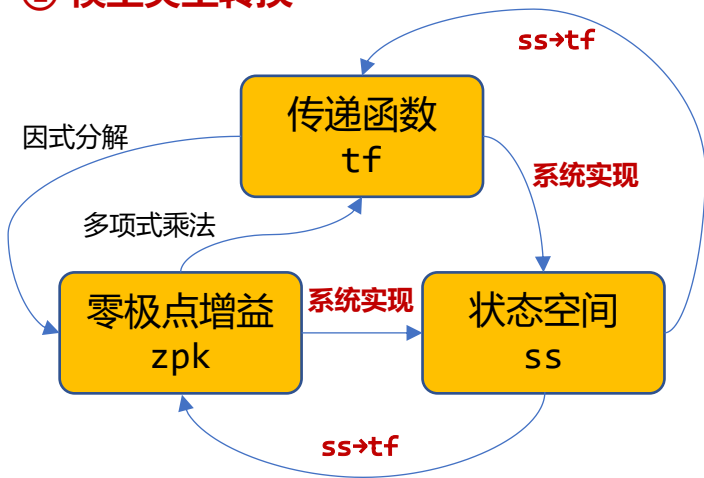
D =
  0.0  0.0
  0.0  0.0
```

连续时间状态空间模型

针对离散时间状态空间模型的创建，同样使用 ss 函数， $G = ss(A,B,C,D,ts)$ ，增加 ts 指定采样时间即可

1. 控制系统的数学模型-模型的转换

① 模型类型转换



系统实现: 对以下形式的传递函数转化为状态空间表达式, 即为系统实现

$$G(s) = \frac{Y(s)}{U(s)} = \frac{b_ms^m + b_{m-1}s^{m-1} + \dots + b_1s + b_0}{a_ns^n + a_{n-1}s^{n-1} + \dots + a_1s + a_0}, n \geq m$$

ss→tf: 状态空间模型到传递函数的转化

$$G(s) = C[sI - A]^{-1}B + D$$

$$\begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \\ \vdots \\ \dot{x}_{n-1} \\ \dot{x}_n \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 & \dots & 0 \\ 0 & 0 & 1 & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \dots & 1 \\ -\frac{a_0}{a_n} & -\frac{a_1}{a_n} & -\frac{a_2}{a_n} & \dots & -\frac{a_{n-1}}{a_n} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_{n-1} \\ x_n \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 0 \\ 1 \end{bmatrix} \cdot u$$
$$y = \begin{bmatrix} \frac{b_0 - a_0 b_n}{a_n} & \frac{b_1 - a_1 b_n}{a_n} & \frac{b_2 - a_2 b_n}{a_n} & \dots & \frac{b_{n-1} - a_{n-1} b_n}{a_n} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_{n-1} \\ x_n \end{bmatrix} + \frac{b_n}{a_n} \cdot u$$

Syslab中可以直接通过 `tf()`、`ss()`、`zpk()` 函数实现不同表达形式模型之间的相互转换

实操9: 将以下状态空间模型转化为传递函数模型

$$\dot{x} = \begin{bmatrix} -2 & -1 \\ 1 & -2 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + \begin{bmatrix} 1 & 1 \\ 2 & -1 \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \end{bmatrix}$$
$$y = \begin{bmatrix} 1 & 0 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + \begin{bmatrix} 0 & 1 \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \end{bmatrix}$$

```
A = [-2 -1; 1 -2]
B = [1 1; 2 -1]
C = [1 0]
D = [0 1]
ltiSys = ss(A, B, C, D);
# 直接使用tf函数进行转换
sys = tf(ltiSys)
```

输入 1 到输出 1
1.0s

1.0s^2 + 4.0s + 5.0

输入 2 到输出 1
1.0s^2 + 5.0s + 8.0

1.0s^2 + 4.0s + 5.0

连续时间传递函数模型

示例10: 将以下零极点增益模型转化为状态空间模型

$$H(s) = \frac{10(s+5)}{(s+8)(s+10)(s+2)}$$

```
z = [-5]
p = [-8, -10, -2]
H = zpk(z, p, 10);
# 直接使用ss函数进行转换
sys = ss(H)
```

```
A =
  0.0    4.0    0.0
  0.0    0.0    8.0
 -5.0  -14.5  -20.0
B =
  0.0
  0.0
  2.0
C =
  0.78125  0.625  0.0
D =
  0.0
```

连续时间状态空间模型

1. 控制系统的数学模型-模型的转换

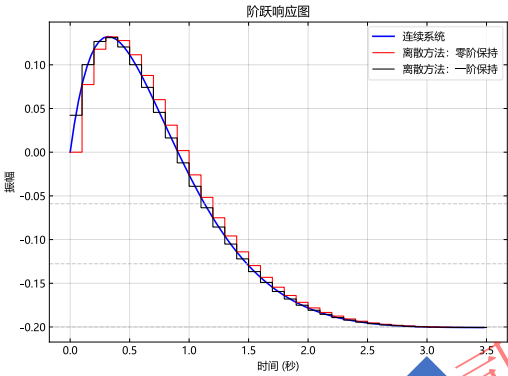
② 连续 - 离散转换

支持连续-离散系统模型的相互转换，以及离散系统之间的重采样

实操11： 将以下连续模型离散化，采样时间为 $t_s = 0.1s$

$$G(s) = \frac{s-1}{s^2+4s+5}$$

```
G = tf([1,-1],[1,4,5])
# 默认离散方法: 零阶保持
Gd = c2d(G,0.1)
# 选用一阶保持离散方法
Gd1 = c2d(G,0.1,:foh)
```



输出

```
julia> Gd
0.07735946566180907z - 0.08556727104741424
-----
1.0z^2 - 1.6292810191076135z + 0.6703200460356392

Sample Time: 0.1 (seconds)
离散时间传递函数模型

julia> Gd1
0.042263385959546824z^2 - 0.010930071156678337z - 0.03954112018847361
-----
1.0z^2 - 1.6292810191076135z + 0.6703200460356392

Sample Time: 0.1 (seconds)
离散时间传递函数模型
```

示例12： 将以下离散时间系统进行重采样，重采样时间为 $t_s = 0.02s$

$$H(z) = \frac{z-1}{z^2+z+0.3}, ts=0.1s$$

```
H = tf([1,-1],[1,1,0.3],0.1)
# 对原系统进行重采样
Hc = d2d(H,0.02)
```

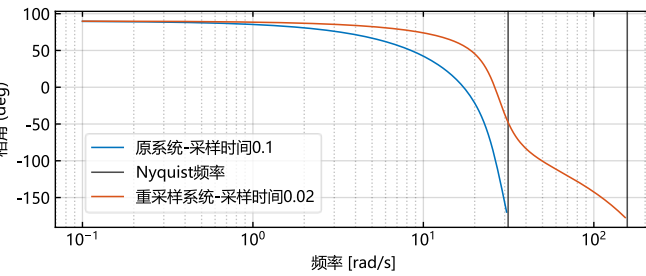
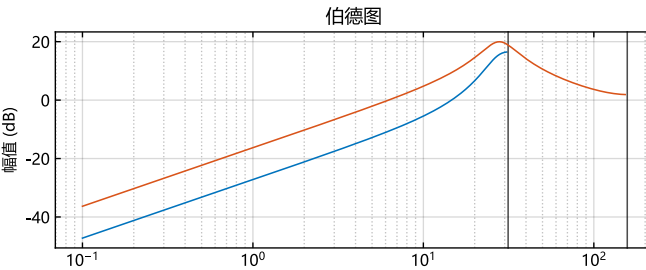
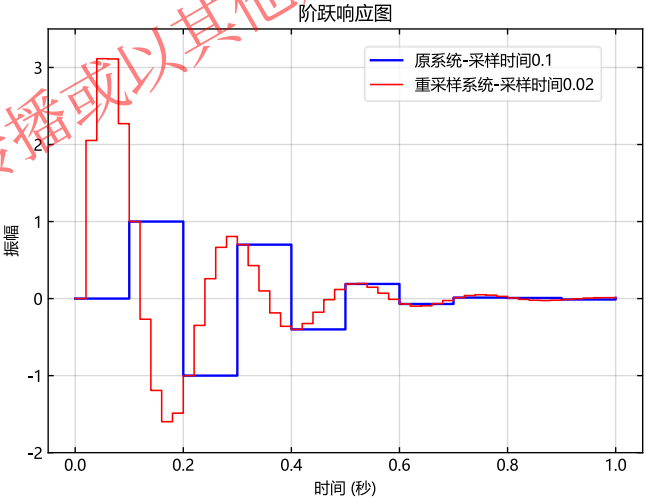
```
julia> H
1.0z - 1.0
-----
1.0z^2 + 1.0z + 0.3

Sample Time: 0.1 (seconds)
离散时间传递函数模型

julia> Hc
2.0527782215874124z - 2.052778221587414
-----
1.0z^2 - 1.5169814293463761z + 0.7860030855966229

Sample Time: 0.02 (seconds)
离散时间传递函数模型
```

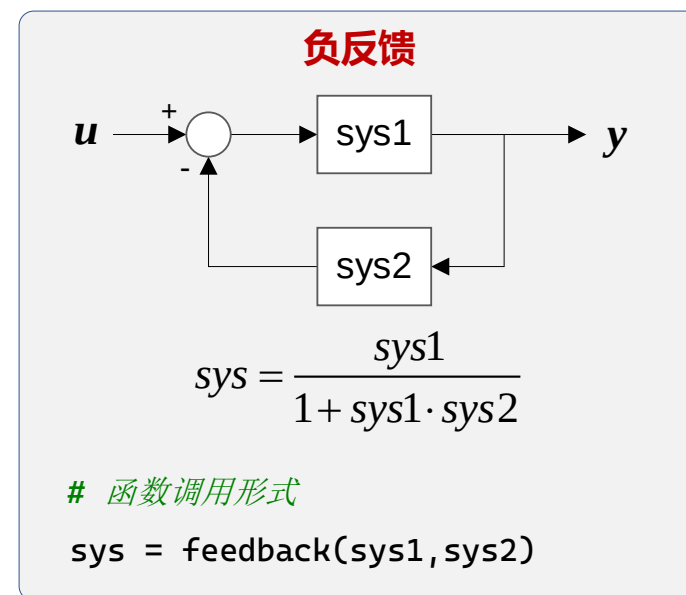
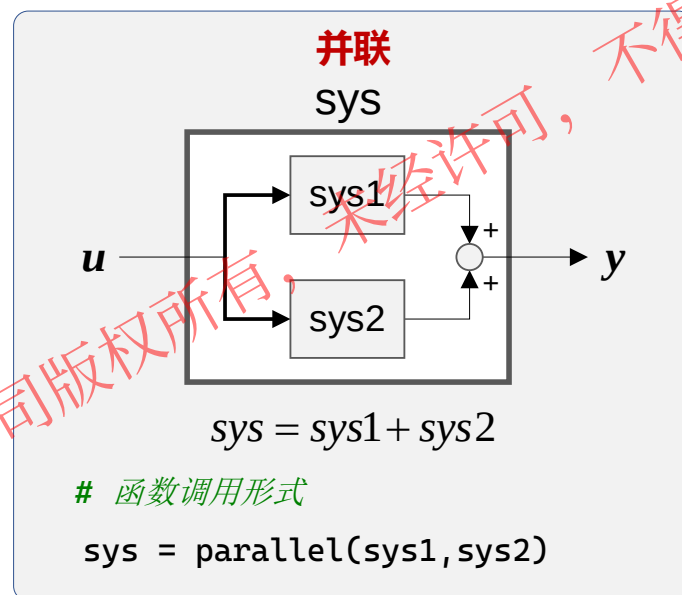
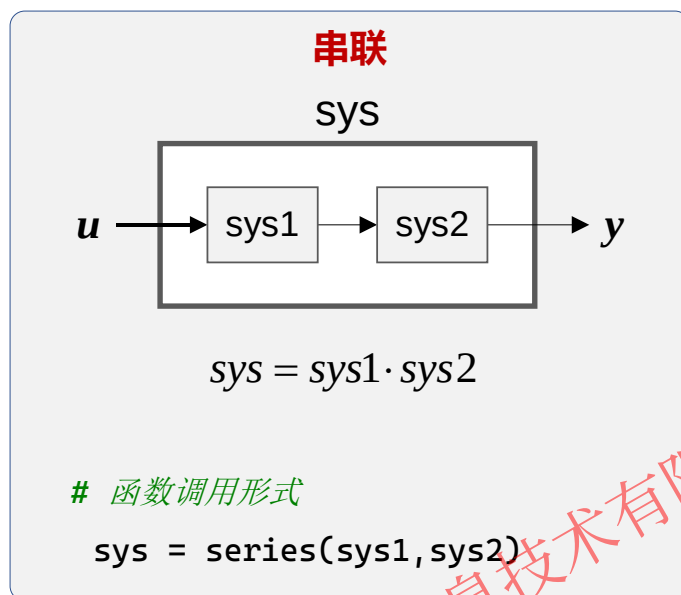
输出



1. 控制系统的数学模型-模型的连接

① 系统连接

- 在实际应用中，整个控制系统由被控对象和控制装置组成，存在多个环节组合而成。每个单一模型都可以用一组微分方程或传递函数来描述
- 模型间连接主要有串联连接、并联连接、串并联连接和反馈连接等。对系统的不同连接情况，可以进行模型的化简



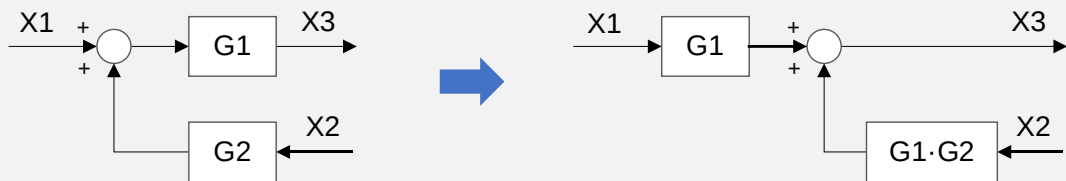
除上述三种典型连接外，Syslab还提供 `append()`、`connect()`、`lft()` 等函数支持一些相对复杂的连接，具体可以参考帮助文档

1. 控制系统的数学模型-模型的连接

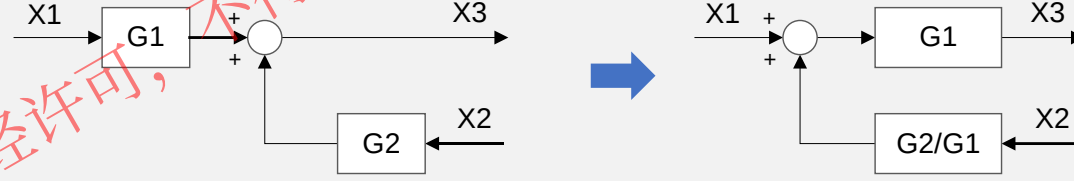
② 系统变换

在复杂结构框图化简中，经常需要将某个支路的输入点从一个节点移动到另一个节点上，进而便于系统推导与简化

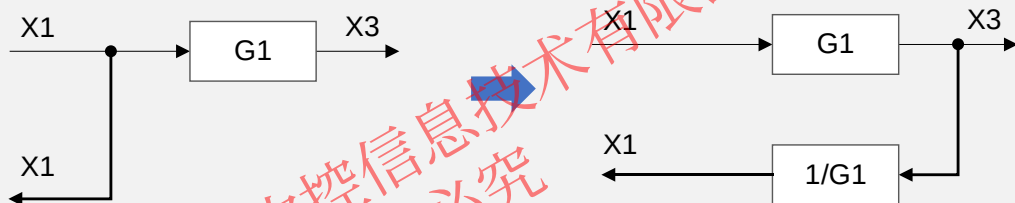
A. 相加点后移等效变换



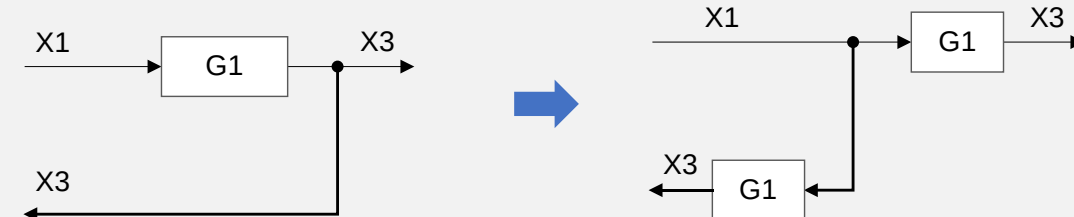
B. 相加点前移等效变换



C. 分支点后移等效变换

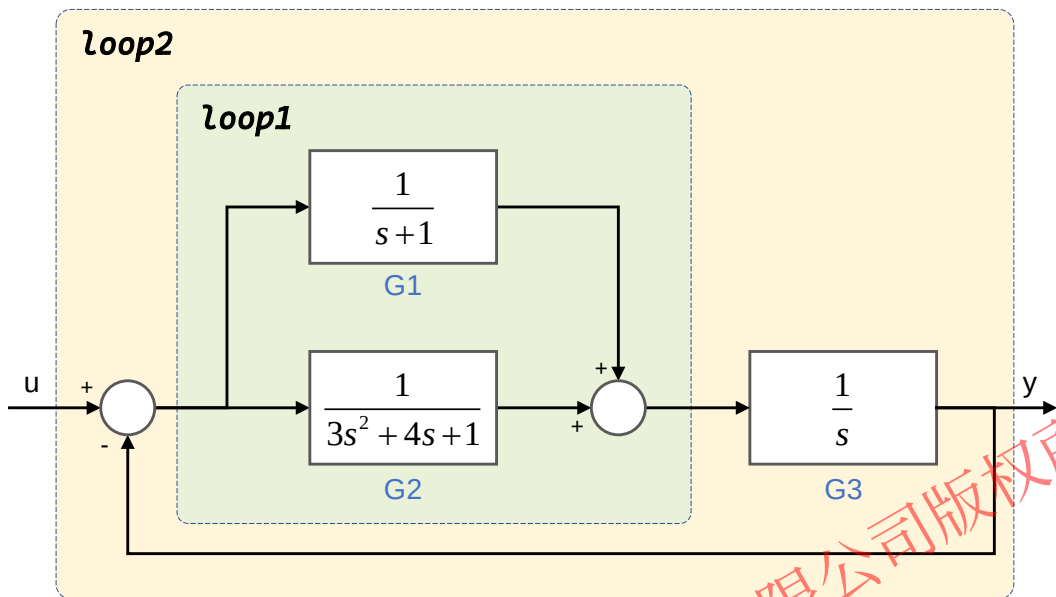


D. 分支点前移等效变换



1. 控制系统的数学模型-模型的连接

示例13: 化简如图系统，求系统的传递函数



```
# 定义各模块模型
G1 = tf([1],[1,1])
G2 = tf([1],[3,4,1])
G3 = tf([1],[1,0])
# 计算loop1
loop1 = parallel(G1,G2)
# 计算loop2
loop2 = feedback(loop1*G3,1)
```

```
julia> loop1
      3s^2 + 5s + 2
-----
3s^3 + 7s^2 + 5s + 1
```

连续时间传递函数模型

```
julia> loop2
      3s^2 + 5s + 2
-----
3s^4 + 7s^3 + 8s^2 + 6s + 2
```

连续时间传递函数模型

1. 控制系统的数学模型-模型的化简

示例14: 考虑以下系统模型 $G(s) = \frac{5s^3 + 50s^2 + 155s + 150}{s^4 + 11s^3 + 41s^2 + 61s + 30}$ 如果不对其进行任何变换, 无法发现该模型有哪些特点

```
G = tf([5,50,155,150],[1,11,41,61,30])
GTrans = zpk(G)
```

$$5.0 \frac{(1.0s + 5.0)(1.0s + 3.000000000000009)(1.0s + 1.999999999999996)}{(1.0s + 4.999999999999996)(1.0s + 3.000000000000004)(1.0s + 1.999999999999982)(1.0s + 1.000000000000009)}$$

连续时间传递函数模型

从零极点模型发现, 系统在 $s = -2$ 、 -3 、 -5 处有近似相同的零极点。在数学上就可以直接进行对消

```
# 系统的最小实现
Gr = minreal(G)
```

$$\frac{5.0}{1.0s + 1.000000000000009}$$

连续时间传递函数模型

示例15: 考虑以下 5 阶双输入双输出系统的最小实现

$$\dot{x} = \begin{bmatrix} -6 & -1.5 & 2 & 4 & 9.5 \\ -6 & -2.5 & 2 & 5 & 12.5 \\ -5 & 0.25 & -0.5 & 3.5 & 9.75 \\ -1 & 0.5 & 0 & -1 & 1.5 \\ -2 & -1 & 1 & 2 & 3 \end{bmatrix} x + \begin{bmatrix} 6 & 4 \\ 5 & 5 \\ 3 & 4 \\ 0 & 2 \\ 3 & 1 \end{bmatrix} u$$
$$y = \begin{bmatrix} 2 & 0.75 & -0.5 & -1.5 & -2.75 \\ 0 & -1.25 & 1.5 & 1.5 & 2.25 \end{bmatrix} x$$

```
G = ss(A,B,C,D)
Gr = minreal(G)
```

A =
-1.2278481012658222 -0.19130639750752176 2.7601986869278203
0.37367324372963995 -1.627513494744153 0.8611642129490126
0.19052992136656932 -0.21798219140272287 -1.144638403990033

B =
-8.888194417315589 -7.200562565926553
0.0 -3.186204440110084
0.0 0.0

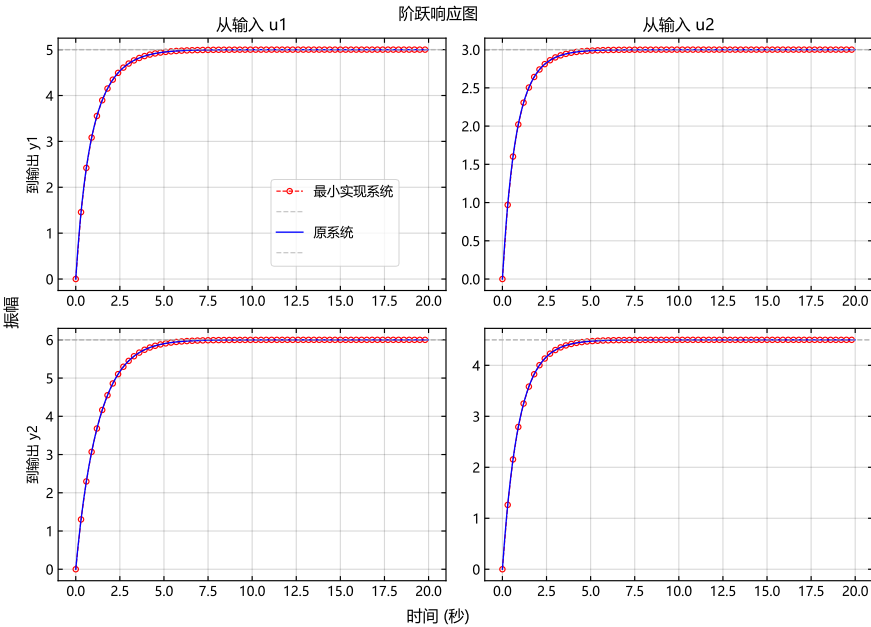
C =
-0.675052740555615 0.2701519974158175 0.4344110957473547
-0.5625439504630119 -0.29796176185568113 -0.08835479913505484

D =
0.0 0.0
0.0 0.0

连续时间状态空间模型

5 阶系统消除了 2 个状态变量, 得到的最小实现模型为 3 阶

原系统与最小实现系统的阶跃响应对比



1. 控制系统的数学模型-模型的化简

在模型化简中，均衡实现是状态方程的一种非常实用的表示形式，该模型可以将各个状态变量在整个控制系统中的重要程度明确表示出来。Syslab控制工具箱提供 `balreal()` 函数可以将已知模型进行均衡实现

示例16：考虑将以下模型进行简化处理

$$G(s)=\frac{s^3+11s^2+36s+26}{s^4+14.6s^3+74.96s^2+153.7s+99.65}$$

①. 将上述模型先进行一个均衡状态空间实现

```
G = tf([1, 11, 36, 26], [1, 14.6, 74.96, 153.7, 99.65])
# 均衡状态空间模型实现
Gb, g = balreal(ss(G))

julia> Gb
A =
-3.601435772897594    0.8212109733891786   -0.6163395682048404    0.058314952589902255
-0.8212109733891763   -0.5929711329634693    1.0273084167890452   -0.09033395280227531
-0.6163395682048548   -1.0273084167890418   -5.913814939296773    1.127164702405639
-0.05831495258988176  -0.09033395280222549  -1.1271647024053315   -4.4917781548421285
B =
-1.001984036600179
-0.106412279753052
-0.08612411663246879
-0.008111706668116852
C =
-1.0019840366001798  0.10641227975305181  -0.08612411663246491  0.008111706668119023
D =
0.0

连续时间状态空间模型

julia> g
4-element Vector{Float64}:
 0.13938496656762903
 0.009548165713944968
 0.0006271217092401152
 7.3244695977053e-6
```

通过 Gramian 矩阵的对角线向量 g，判定均衡实现的后 2 个状态对系统影响较小

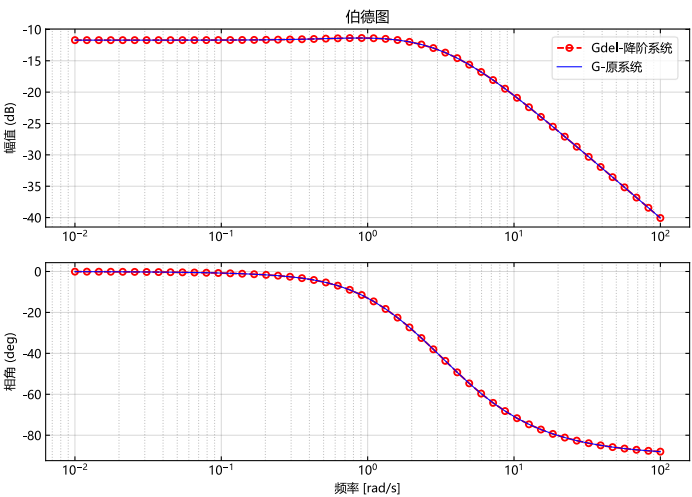
②. 通过 `modred()` 函数从均衡实现中消除对系统影响较小的状态

```
# 消除影响较小的两个状态
Gdel = modred(Gb, [3,4], method=:Truncate)

julia> Gdel
A =
-3.601435772897594    0.8212109733891786
-0.8212109733891763   -0.5929711329634693
B =
-1.001984036600179
-0.106412279753052
C =
-1.0019840366001789  0.10641227975305181
D =
0.0

连续时间状态空间模型
```

消除2个影响小的状态变量后得到的系统模型



原始 4 阶系统与简化的 2 阶系统在频率上的响应几乎一致



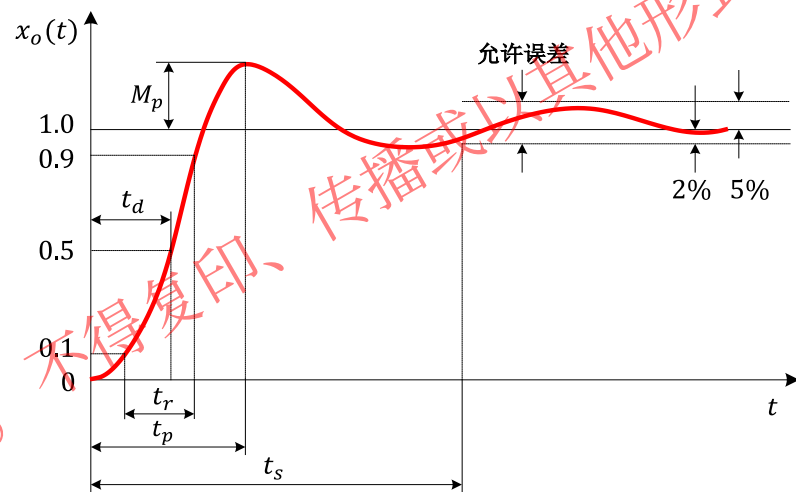
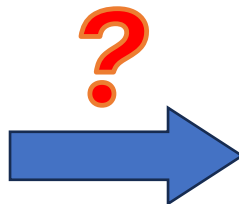
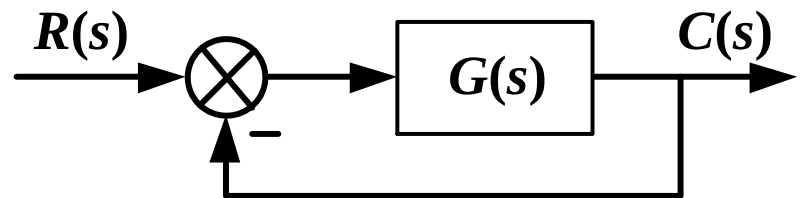
目录

1. 控制系统的数学模型

2. 控制系统的时域分析

3. 控制系统的频域分析

如何开展时域分析?



已经构建好系统模型，如何开展时域分析？

1. **如何计算**并可视化系统时域响应？
2. **如何获取**系统时域响应相关指标？
3. **如何对比**分析不同系统响应特性？

■ 时域响应

- 阶跃响应、脉冲响应、斜坡响应、自定义响应

■ 数据及指标获取

- 阶跃响应数据、脉冲响应数据、斜坡响应数据、自定义响应数据
- 工具箱交互式查看

■ 特性对比

2. 控制系统的时域分析-阶跃信号响应

时域分析是一种最直观、最直接的分析。一般可以为控制系统预先规定一些特殊的试验输入信号，然后比较各种系统对这些信号的响应情况。

经常采用的试验输入信号

信号名称	时域表达式	复频域表达式
单位阶跃	$1(t), t \geq 0$	$\frac{1}{s}$
单位斜坡	$t, t \geq 0$	$\frac{1}{s^2}$
单位加速度	$\frac{1}{2}t^2, t \geq 0$	$\frac{1}{s^3}$
单位脉冲	$\delta(t), t = 0$	1
正弦函数	$A \sin \omega t$	$\frac{A\omega}{s^2 + \omega^2}$

一. 阶跃响应 – step()

- 对于稳定系统，通常在系统阶跃响应曲线上来定义系统动态性能指标
- 系统的单位阶跃响应不仅完整反映了系统的动态特性，而且反映了系统在单位阶跃信号输入下的稳定状态。同时，单位阶跃信号又是一个最简单、最容易实现的信号。

step调用方式	说明
step(sys)	计算并直接返回系统阶跃响应图。其中sys可以是：tf、ss、zpk
step(sys,t)	计算向量 t 指定时间内的阶跃响应。t为标量：计算[0,t]内的响应，t为向量，计算各点的阶跃响应。示例： - step(G,5) - step(G,0:0.1:10)
step(____,fmt)	计算并直接返回系统阶跃响应图。fmt为绘图样条属性设置字符串。示例： step(G,"-bo",linewidth = 1,markersize = 5,...)
y,t,x = step(sys, t, fig = false)	计算阶跃响应数据，y 为响应数据数组，t 为时间向量，x 为状态数组

2. 控制系统的时域分析-阶跃信号响应

阶跃响应 – step()

实操1: 计算并绘制以下系统的阶跃响应 $G(s) = \frac{2s + 25}{s^2 + 4s + 25}$

```
G = tf([2 25],[1 4 25])  
# 阶跃响应基础绘图  
step(G, 3.5)
```

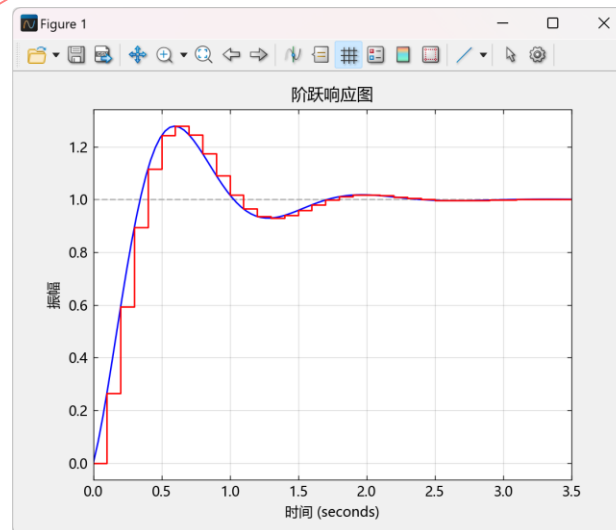
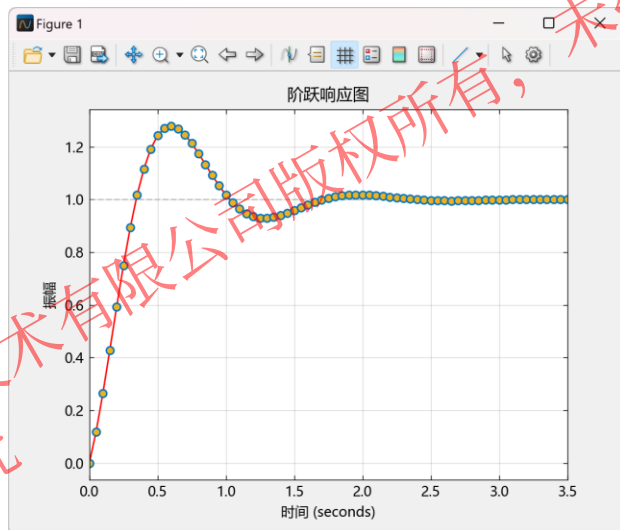
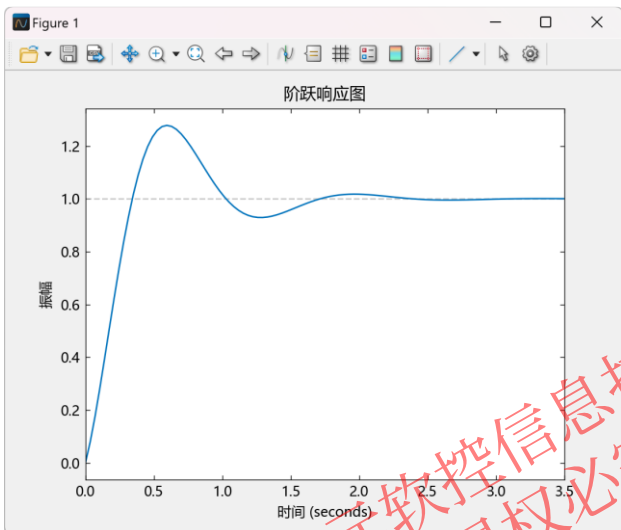
```
# 图形修饰  
step(G, 0:0.05:3.5, "-ro", linewidth = 1,  
      markersize = 5, markeredgecolor =  
      "#0072BD", markerfacecolor = "#EDB120")  
grid("on")
```

```
# 多个系统响应绘图叠加  
step(G, 3.5, "-b")  
hold("on")  
step(c2d(G, 0.1), 3.5, "-r")  
grid("on")
```

```
# 直接获取阶跃响应数据  
y,t = step(G, 3.5, fig = false)
```

```
julia> t  
0.0:0.035:3.5
```

```
julia> y  
1×101 Matrix{Float64}:  
 0.0  0.0795734  0.174988  
 0.281454  0.394494  0.510026  
 0.624432  0.734596  0.837934 ...  
 1.00076  1.00093  1.00105  
 1.00112  1.00115  1.00115  
 1.00111  1.00105  1.00096  
 1.00086
```



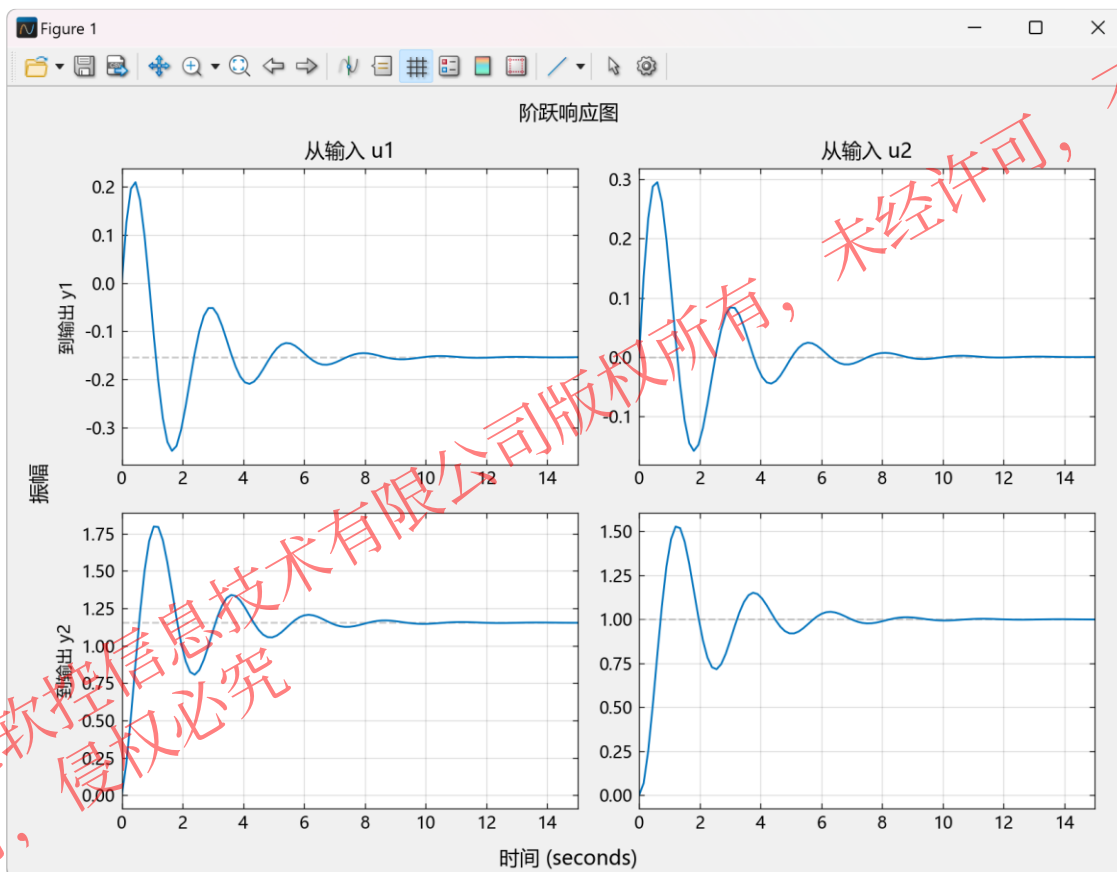
2. 控制系统的时域分析-阶跃信号响应

阶跃响应 – step()

示例2: 计算并绘制双输入双输出系统的阶跃响应

$$\begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \end{bmatrix} = \begin{bmatrix} -1 & -1 \\ 6.5 & 0 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \end{bmatrix}$$
$$\begin{bmatrix} y_1 \\ y_2 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \end{bmatrix}$$

```
# 定义系统矩阵
A = [-1 -1; 6.5 0]
B = [1 1; 1 0]
C = [1 0; 0 1]
D = zeros(2,2)
# 创建状态空间模型
G = ss(A,B,C,D)
# 计算系统阶跃响应
step(G, 15)
fig = gcf()
ax = fig.axes
grid.(ax, "on")
```



```
# 获取输出结构体数据
y,t = step(G,15,fig = false)
```

```
julia> t
101-element Vector{Float64}:
 0.0:0.15:15.0
```

```
julia> y
2×501×2 Array{Float64, 3}:
[:, :, 1] =
 0.0  0.125343  0.19577  0.209482  0.172538  0.0969517...
 0.0  0.215268  0.526474  0.878502  1.21844  1.50235...

[:, :, 2] =
 0.0  0.135923  0.234677  0.28819  0.295585  0.262293...
 0.0  0.0687662  0.252891  0.511603  0.799804  1.07472...
```

对于MIMO系统的响应数据 y 为一个**三维数组**，其维度为：

$N_y * N * N_u$ ，其中：

- N_y 为系统输出数量
- N 为时间向量长度
- N_u 为系统输入数量

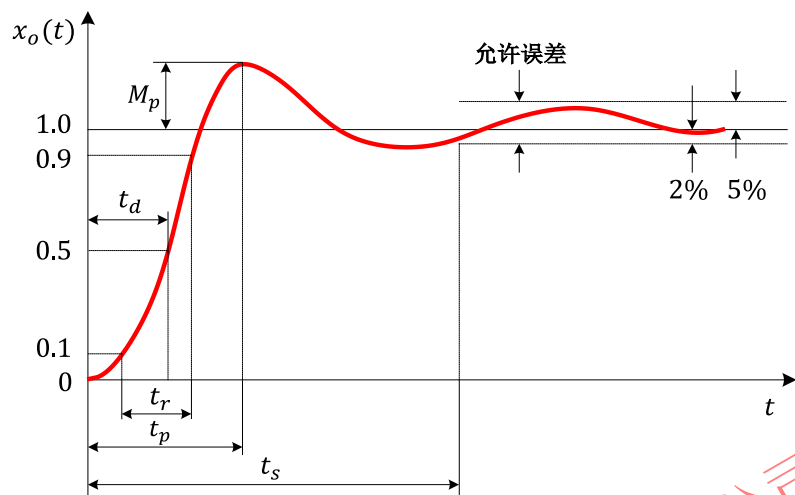
因此，通过 $y[i,:,j]$ 可取出第 j 个输入到第 i 个输出的阶跃响应数据向量。

2. 控制系统的时域分析-阶跃信号响应

阶跃响应 – step()

时域分析性能指标

通常控制系统的性能指标以系统对单位阶跃输入量的瞬态响应形式给出

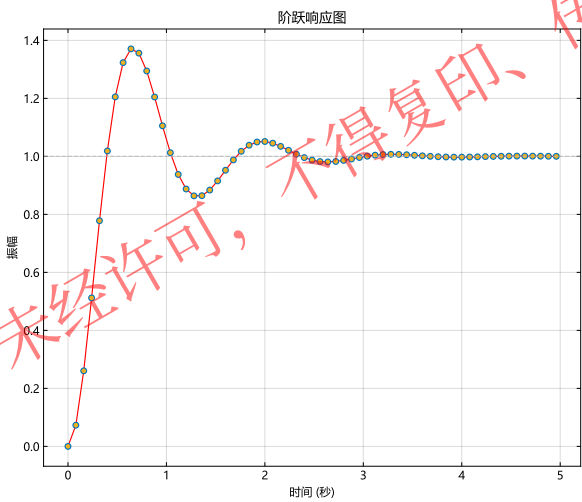


时域性能指标	含义
上升时间 t_r	响应曲线由稳态值的10%上升到稳态值得90%所需的时间
峰值时间 t_p	响应曲线从零上升到第一个峰值所需要的时间
最大超调量 M_p	响应曲线最大峰值与稳态值之差
调整时间 t_s	响应曲线达到并一直保持在允许误差范围内的最短时间
延迟时间 t_d	响应曲线从0上升到稳态值的50%所需要的时间

通过 `stepinfo()` 函数计算并获取动态系统阶跃响应特性

实操4：获取系统的阶跃响应特性

$$G(s) = \frac{25}{s^2 + 3s + 25}$$



更多设置选项，参考帮助文档

```
G = tf([25],[1 3 25])
# 计算并获取系统阶跃响应特性
res = stepinfo(G)
# 上升时间
res.RiseTime
# 最大超调
res.Overshoot
# 峰值
res.Peak
# 峰值时间
res.PeakTime
# 调整时间
res.SettlingTime
```

输出

```
julia> # 上升时间
julia> res.RiseTime
1×1 Matrix{Float64}:
 0.2647250314828522

julia> # 最大超调
julia> res.Overshoot
1×1 Matrix{Float64}:
 37.1410271661408

julia> # 峰值
julia> res.Peak
1×1 Matrix{Float64}:
 1.371410271661408

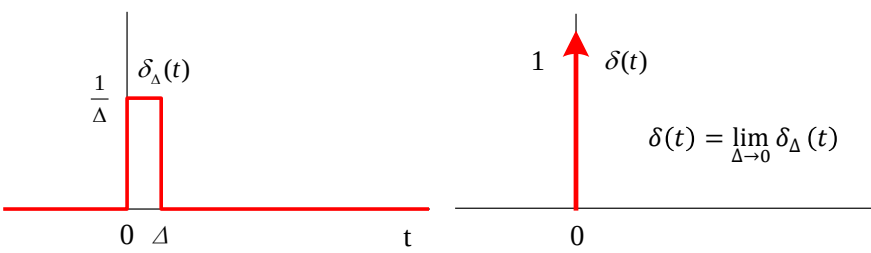
julia> # 峰值时间
julia> res.PeakTime
1×1 Matrix{Float64}:
 0.6447238260382373

julia> # 调整时间
julia> res.SettlingTime
1×1 Matrix{Float64}:
 2.246034069329275
```

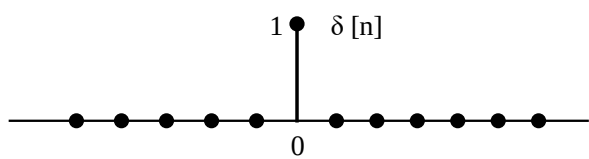
2. 控制系统的时域分析-脉冲信号响应

脉冲响应 – impulse()

系统的脉冲响应（或称为冲激响应）可以用 `impulse()` 函数进行计算并绘制脉冲响应图。



连续时间单位脉冲信号是一个持续时间为 Δ 的短脉冲，对于任意的 Δ 值，其面积均为1，随着 $\Delta \rightarrow 0$ ， $\delta_A(t)$ 变得越来越窄，越来越高，但单位面积不变



离散时间单位脉冲信号可以表达为：

$$u[n] = \begin{cases} 0, & n \neq 0 \\ 1, & n = 0 \end{cases}$$

impulse 调用方式	说明
<code>impulse(sys)</code>	计算并直接返回系统脉冲响应图。其中sys可以是：tf、ss、zpk
<code>impulse (sys,t)</code>	计算向量 t 指定时间内的脉冲响应。t为标量：计算[0,t]内的响应，t为向量，计算各点的脉冲响应。示例： <code>impulse(G,5)</code> <code>impulse(G,0:0.1:10)</code>
<code>impulse(____,fmt)</code>	计算并直接返回系统脉冲响应图。fmt为绘图样条属性设置字符串。示例： <code>impulse(G,"-bo",linewidth = 1,markersize = 5,...)</code>
<code>y,t,x = impulse (sys, t, fig = false)</code>	计算阶跃响应数据，y 为响应数据数组，t 为时间向量，x 为状态数组

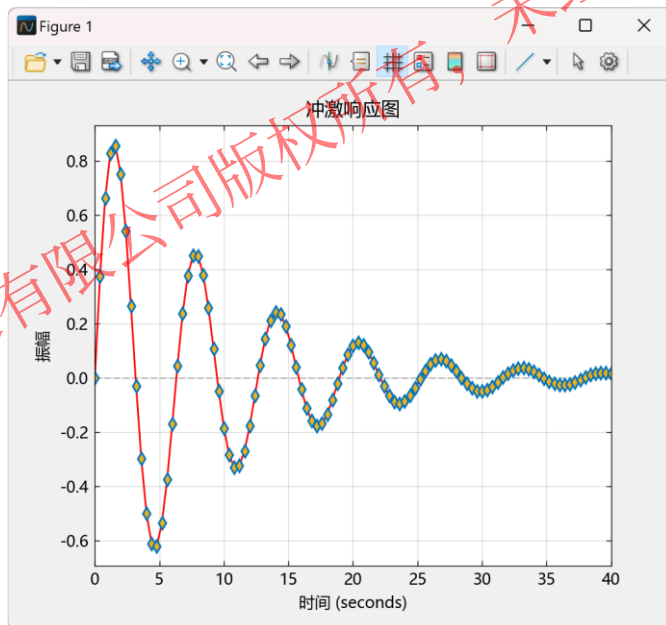
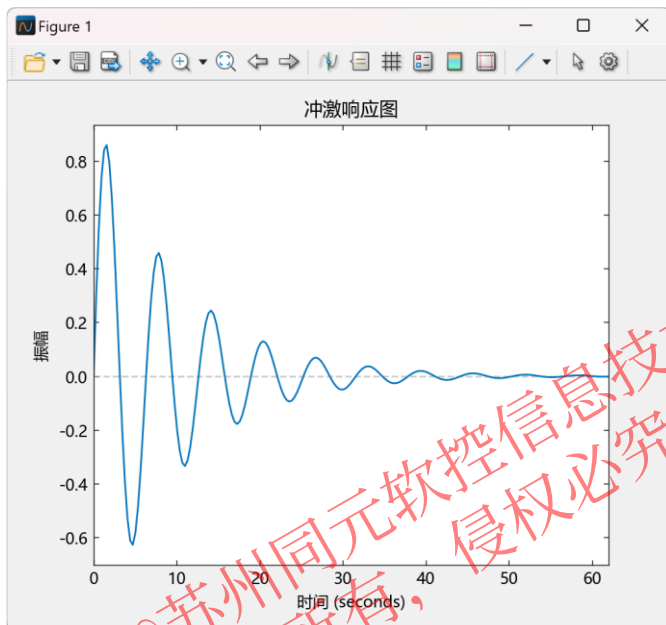
2. 控制系统的时域分析-脉冲信号响应

脉冲响应 - impulse()

示例3: 计算以下系统的脉冲响应 $H(s) = \frac{1}{s^2 + 0.2s + 1}$

```
H = tf([1],[1,0.2,1])  
impulse(H)
```

```
# 图形修饰  
H = tf([1],[1,0.2,1])  
impulse(H,0:0.4:40,"-rd",linewidth = 1,  
markersize = 5, markeredgecolor =  
"#0072BD", markerfacecolor = "#EDB120")  
grid("on")
```



求脉冲响应的另一种方法

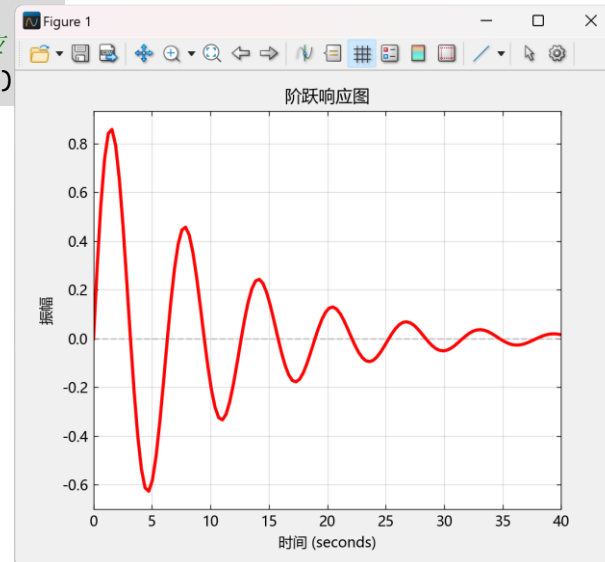
考虑系统的脉冲响应表达式为: $Y(s) = G(s) \cdot U(s)$, 其中脉冲信号 $U(s) = 1$

$$Y(s) = G(s) \cdot 1 = s \cdot G(s) \cdot \frac{1}{s}$$

因此, 求取 $G(s)$ 的脉冲响应, 可以转化为求取 $s \cdot G(s)$ 的单位阶跃响应

示例4: 针对示例21, 通过阶跃函数求取其脉冲响应

```
s = tf('s')  
H = tf([1],[1,0.2,1])  
# 通过阶跃函数step求取 H 的脉冲响应  
step(s*H,40,"-r",linewidth = 1)
```



两种计算方式等价

2. 控制系统的时域分析-斜坡信号响应

斜坡信号响应

Syslab控制工具箱没有提供斜坡信号响应函数，同样可以考虑使用上面的方法进行求取。考虑系统的斜坡响应表达式为： $Y(s) = G(s) \cdot U(s)$ ，其中斜坡信号： $U(s) = \frac{1}{s^2}$

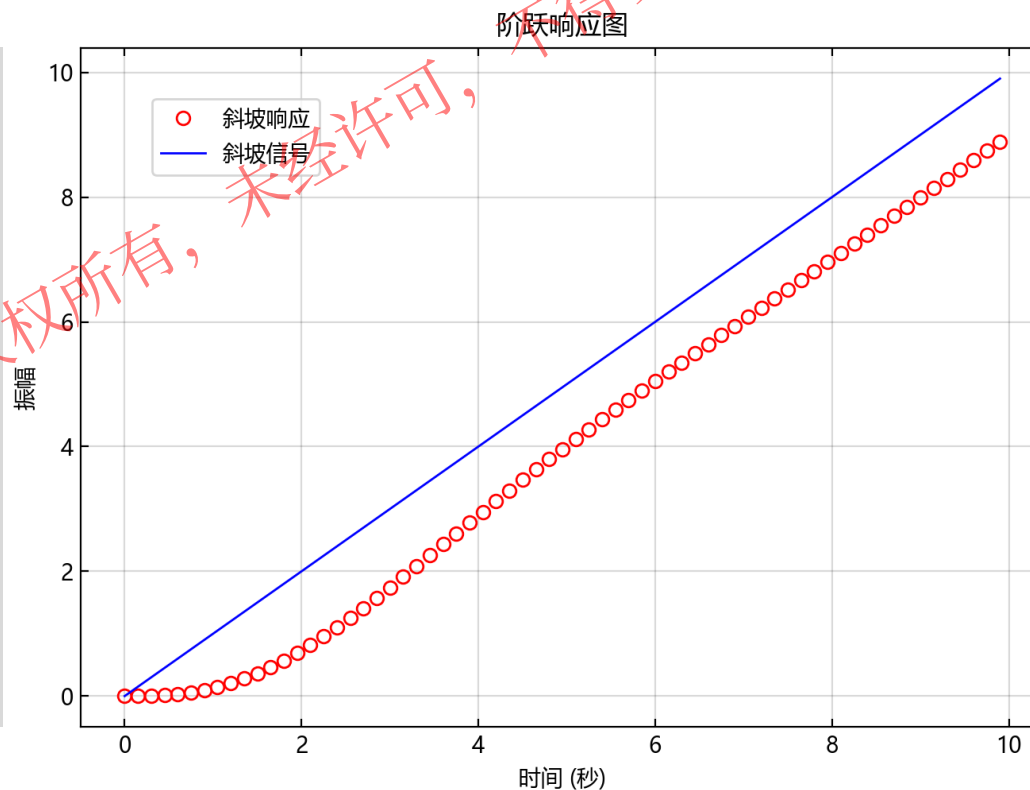
$$Y(s) = G(s) \cdot \frac{1}{s^2} = \frac{1}{s} \cdot G(s) \cdot \frac{1}{s}$$

求取 $G(s)$ 的斜坡响应，转化为求取 $\frac{1}{s} \cdot G(s)$ 的单位阶跃响应

示例5：计算以下系统的斜坡响应

$$\begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ -1 & -1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + \begin{bmatrix} 0 \\ 1 \end{bmatrix} u$$
$$y = \begin{bmatrix} 1 & 0 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

```
# 定义Laplace算子
s = tf('s')
# 定义状态空间矩阵
A = [0 1; -1 -1]
B = [0; 1]
C = [1 0]
D = [0]
# 创建系统模型
G = ss(A,B,C,D)
# 指定时间向量
t = 0:0.15:10
# 通过step计算斜坡响应
step((tf(G)/s), t, "ro")
hold("on")
# 绘制斜坡输入信号
plot(t, t, "-b")
legend("斜坡响应", "斜坡信号")
grid("on")
```



类似的，系统对单位加速度信号的响应可以使用同样的方式计算得到

2. 控制系统的时域分析-自定义信号响应

对任意信号的响应 – lsim()

为了求对任意输入信号的响应，可以使用 `lsim()` 函数

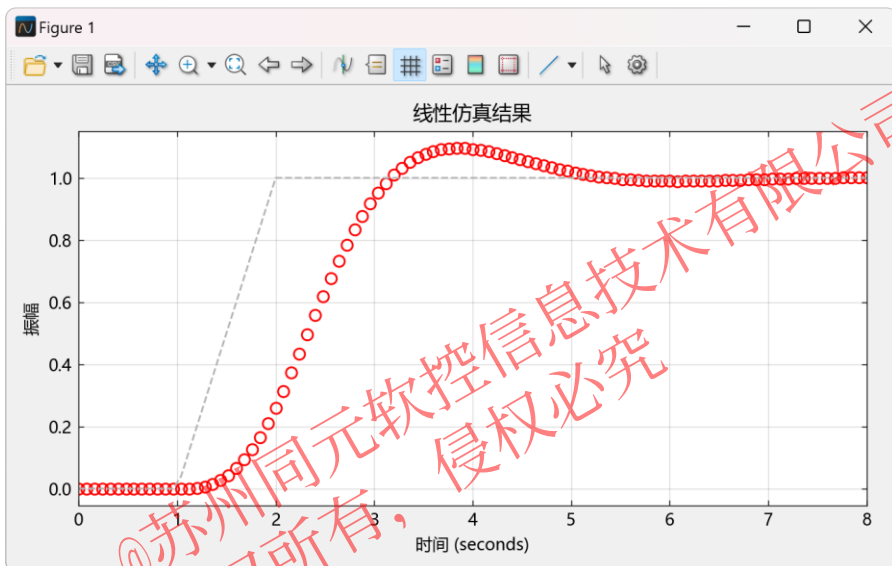
lsim 调用方式	说明
<code>lsim(sys,u,t)</code>	计算并直接返回系统对输入信号 (t,u) 的时域响应图。其中 t 为时间向量, u 的维度为: $N_u \times \text{length}(t)$
<code>lsim(sys,u,t,fmt)</code>	计算并直接返回系统对输入信号 (t,u) 的时域响应图。fmt为绘图样条属性设置字符串。示例: • <code>lsim(G,u,t,"-bo",linewidth = 1,markersize = 5,...)</code>
<code>lsim(sys,u,t,x0 = value)</code>	当sys是状态空间模型时，可以进一步指定初始状态值 x0 , 注意 x0 为向量形式。
<code>lsim(sys,u,t,x0 = value, fmt)</code>	当sys是状态空间模型时，可以进一步指定初始状态值 x0 , fmt为绘图样条属性设置字符串。
<code>res = lsim (sys, u, t, fig = false)</code>	计算任意信号响应数据，生成 res 响应数据为SimResult结构体，不出图。其中： • res.t:时间向量 • res.y:响应数组
<code>y,t,x = lsim (sys, u, t, fig = false)</code>	计算任意信号响应数据，y 为响应数据数组，t 为时间向量，x 为状态数组

2. 控制系统的时域分析-自定义信号响应

对任意信号的响应 - lsim()

示例6: 计算系统对自定义斜坡阶跃信号的响应, 输入信号在 $t = 0$ 时从 0 开始, 在 $t = 1$ 时从 0 开始单位斜坡 1s 到 1, 然后在 1 处保持稳定。

```
# 定义系统
sys = tf([3],[1,2,3])
# 创建输入信号
t = 0:0.08:8
u = max.(0,min.(t.-1,1))
# 计算系统响应
lsim(sys, u, t,"ro")
```



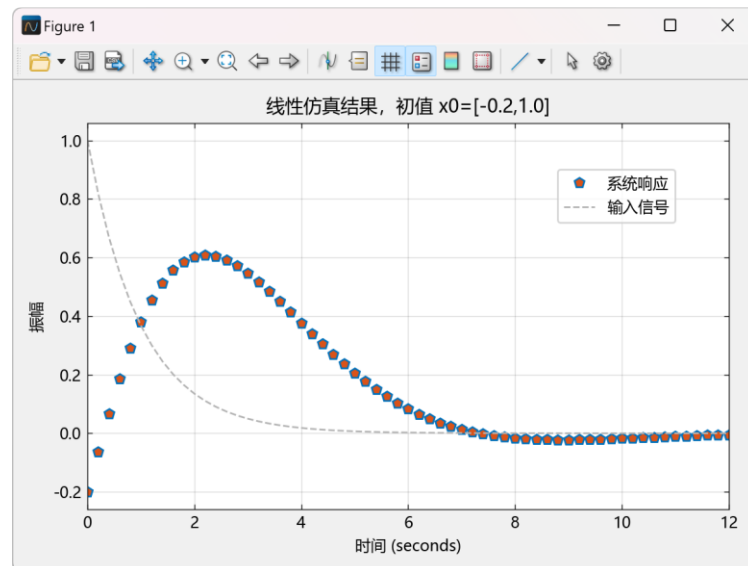
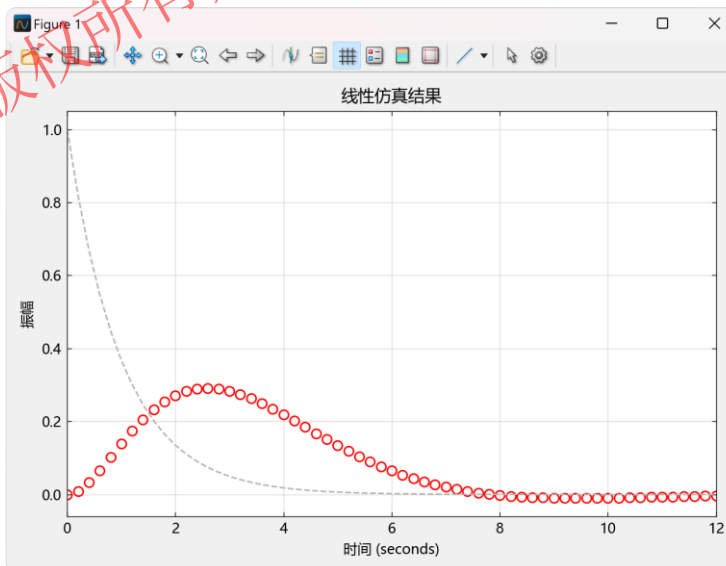
示例7: 考虑以下系统在输入信号 $u = e^{-t}$ 作用下的响应情况, 假设初始状态为 $x(0) = 0$

```
A = [-1 0.5;-1 0]
B = [0;1]
C = [1 0]
D = 0
G = ss(A,B,C,D)
# 创建输入信号
t = 0:0.2:12
u = exp.(-t)
lsim(G,u,t,"ro")
```

状态初值修改

改变初始状态为: $x(0) = [-0.2, 1.0]$

```
lsim(G,u,t,[-0.2,1.0],"rp",markeredgecolor =
"#0072BD", markerfacecolor = "#D95319")
legend("系统响应","输入信号")
title("线性仿真结果, 初值 x0=[-0.2,1.0]")
grid("on")
```



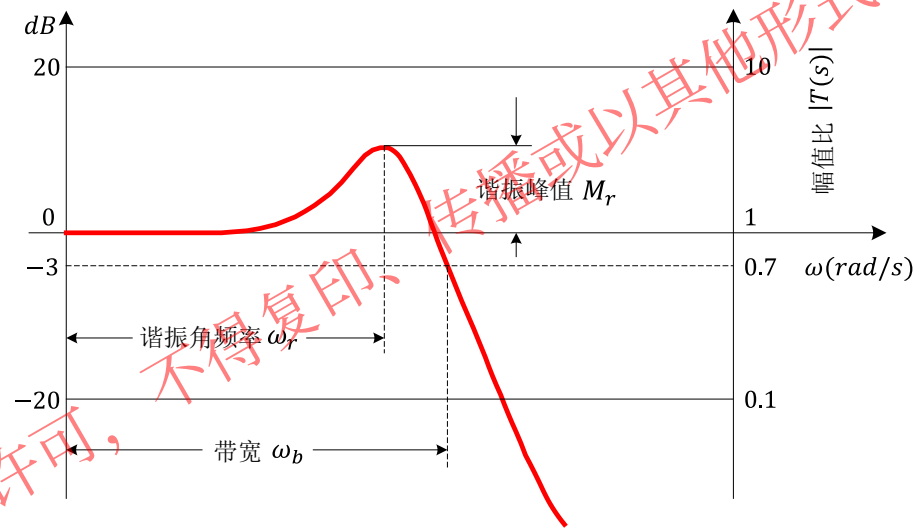
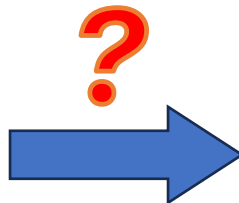
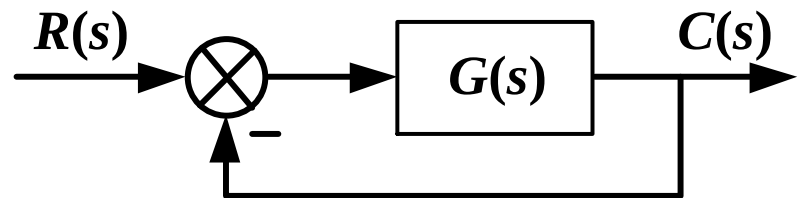
目录

1. 控制系统的数学模型

2. 控制系统的时域分析

3. 控制系统的频域分析

如何开展频域分析?



已经构建好系统模型，如何开展频域分析？

1. **如何计算**并可视化系统频域响应？
2. **如何获取**系统频域响应相关指标？
3. **如何对比**分析不同系统响应特性？

■ 频率响应特性图

- 伯德图、奈奎斯特图、尼克尔斯图

■ 数据及指标获取

- 伯德图数据、奈奎斯特图数据、尼克尔斯图数据
- 工具箱交互式查看

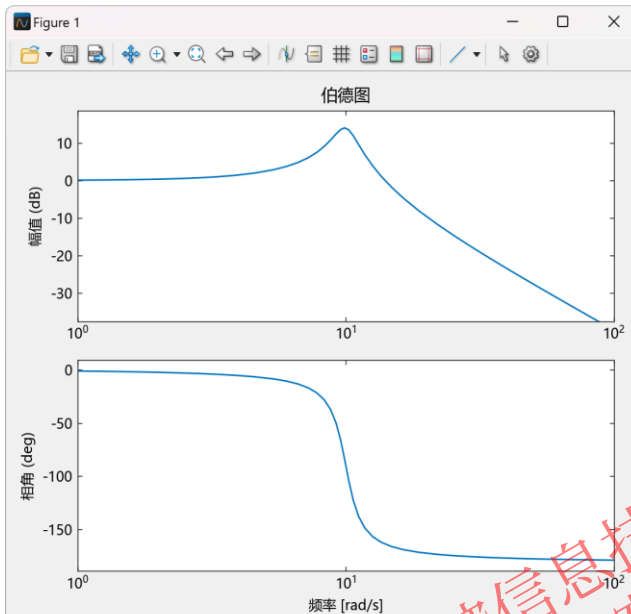
■ 特性对比

3. 控制系统的频域分析-伯德图

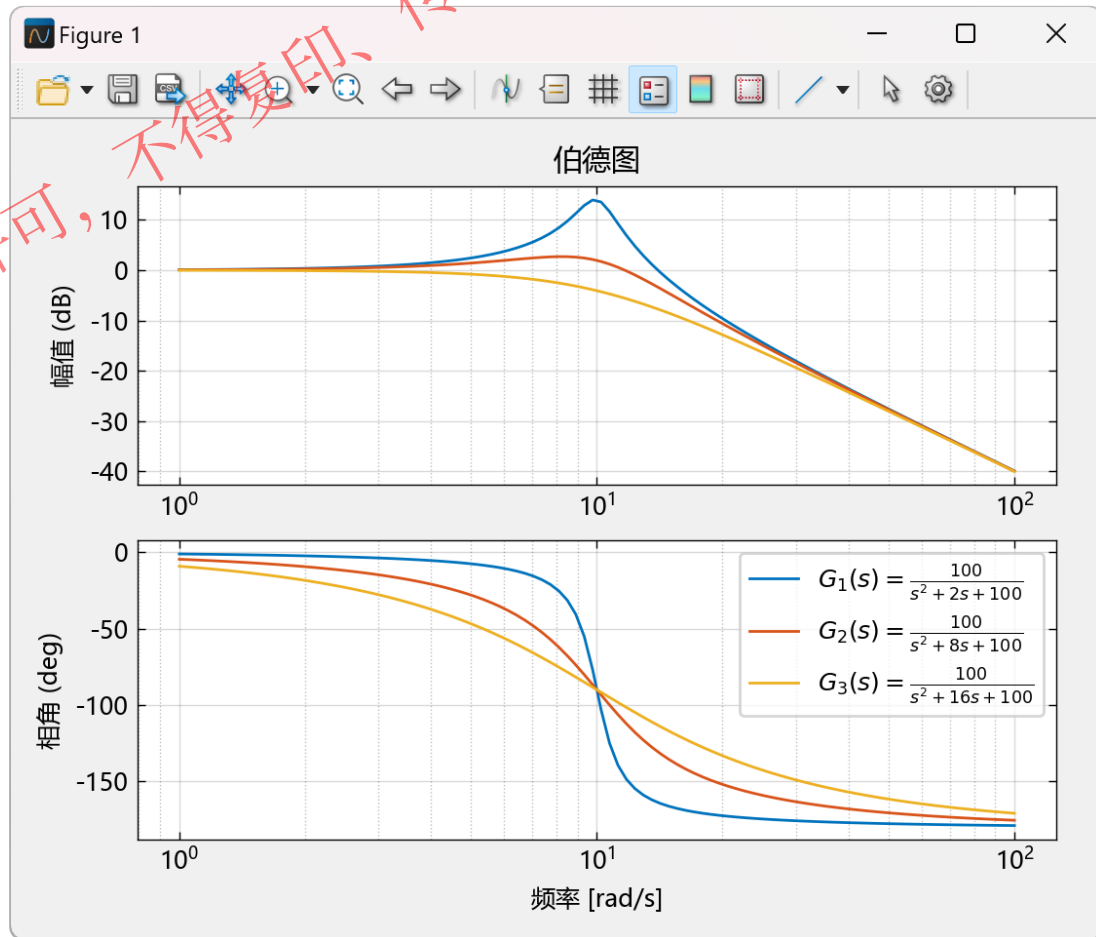
伯德图 - bode()

示例1: 计算系统频率响应并绘制bode图 $G(s) = \frac{100}{s^2 + 2s + 100}$

```
G = tf([100],[1,2,100])  
bode(G)
```



```
# 定义频率向量  
w = logspace(0,2,100)  
# 定义不同阻尼比的系统模型  
G1 = tf([100],[1,2*0.1*10,100])  
G2 = tf([100],[1,2*0.4*10,100])  
G3 = tf([100],[1,2*0.8*10,100])  
# bode图绘制  
bode(G1,w,linewidth = 1)  
hold("on")  
bode(G2,w,linewidth = 1)  
bode(G3,w,linewidth = 1)  
# bode图网格开启  
bodegrid(true)  
# 定义legend  
G1l = raw"$G_1\left(s\right) = \frac{100}{s^2 + 2s + 100}$"  
G2l = raw"$G_2\left(s\right) = \frac{100}{s^2 + 8s + 100}$"  
G3l = raw"$G_3\left(s\right) = \frac{100}{s^2 + 16s + 100}$"  
legend(G1l,G2l,G3l)
```



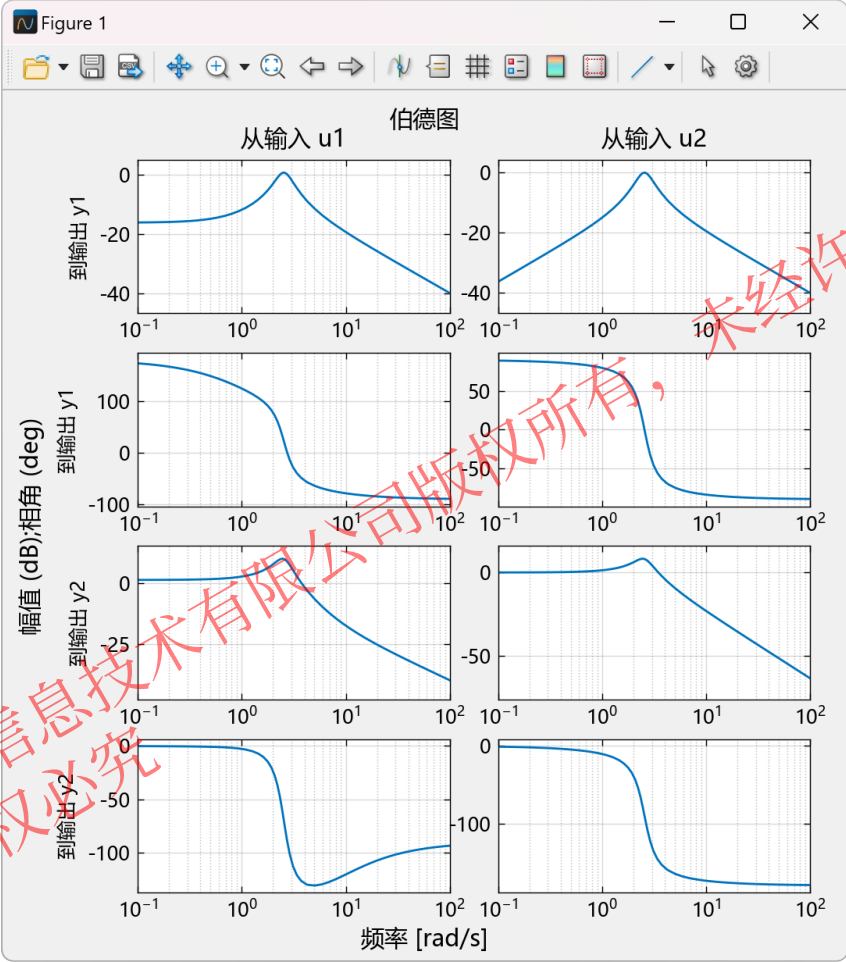
3. 控制系统的频域分析-伯德图

伯德图 – bode()

示例2：计算下面MIMO系统频率响应并绘制bode图

$$\begin{aligned} \begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \end{bmatrix} &= \begin{bmatrix} -1 & -1 \\ 6.5 & 0 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \end{bmatrix} \\ \begin{bmatrix} y_1 \\ y_2 \end{bmatrix} &= \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \end{bmatrix} \end{aligned}$$

```
# 定义系统矩阵
A = [-1 -1; 6.5 0]
B = [1 1; 1 0]
C = [1 0; 0 1]
D = zeros(2,2)
# 创建状态空间模型
G = ss(A,B,C,D)
# 绘制系统bode图
bode(G)
bodegrid(true)
```



计算并获取系统频率响应数据

```
w = logspace(-2,3,100)
mag, phase, wout = bode(G,w,fig = false)
```

```
julia> mag
2x2x100 Array{Float64, 3}:
[:, :, 1] =
 0.153856 0.00153848
 1.15386  1.00001
[:, :, 2] =
 0.153859 0.00172822
 1.15387  1.00002
[:, :, 3] =
 0.153862 0.00194136
 1.15387  1.00002
... ..
```

```
julia> phase
2x2x100 Array{Float64, 3}:
[:, :, 1] =
 179.339      89.9119
 -0.0117543  -0.0881486
[:, :, 2] =
 179.257      89.901
 -0.0132043  -0.0990199
[:, :, 3] =
 179.166      89.8888
 -0.0148333  -0.111232
... ..
```

系统频率响应数据mag、phase均为**三维数组**

其维度： $N_y * N_u * length(wout)$ ，其中：

- N_y 为系统输出数量
- N_u 为系统输入数量
- $length(wout)$ 为频率向量长度

因此，通过 `mag[i,j,:]` 可取出第 **j** 个输入到第 **i** 个输出的幅频响应数据向量

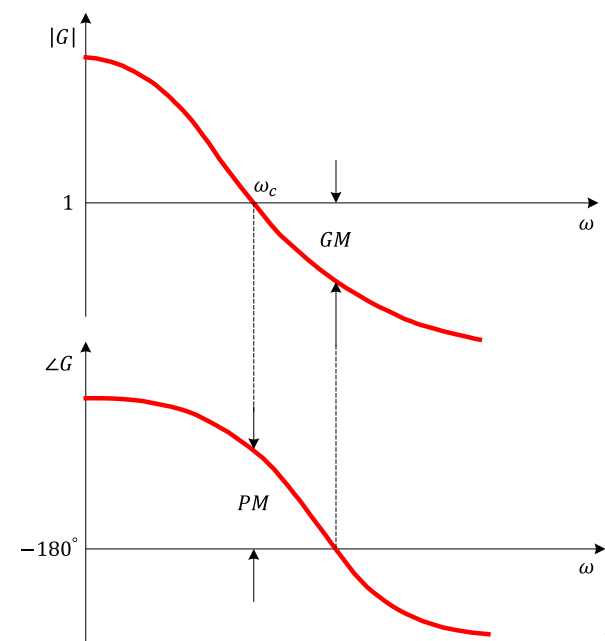


3. 控制系统的频域分析-伯德图

伯德图 – bode()

系统频域指标

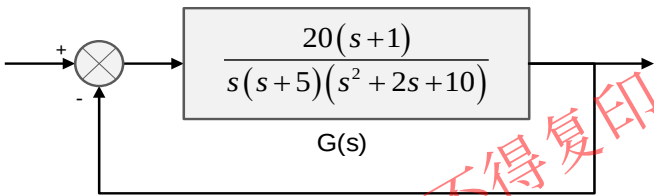
①. 开环频域指标



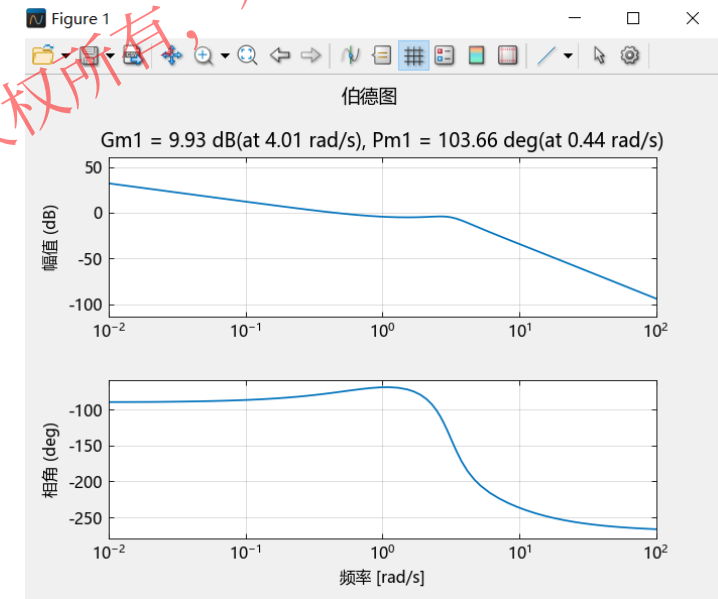
开环频域指标	含义
开环剪切频率 ω_c	开环频率特性幅值为1对应的频率
幅值裕度 GM	在相角等于 -180° 的频率上，幅频特性 $ G(j\omega) $ 的倒数
相位裕度 PM	剪切频率处，相频特性距 -180° 线的相位差

通过 margin() 函数计算系统开环频率指标

示例3：设一闭环系统如下，绘制其开环传递函数bode图，并确定其增益裕度、相位裕度



```
s = tf('s')
Gopenloop = 20*(s+1)/(s*(s+5)*(s^2+2*s+10))
margin(Gopenloop)
```



```
Gm, Pm, Wcg, Wcp = margin(Gopenloop, fig = false)
```

```
julia> Gm #绝对单位，可通过 20*log10.(Gm)计算dB值
1×1 Matrix{Float64}:
 3.1369449122043203

julia> Pm # 相位裕度
1×1 Matrix{Float64}:
 103.6573039145391

julia> Wcg # 增益裕度对应频率
1×1 Matrix{Float64}:
 4.013193642825267

julia> Wcp # 开环剪切频率
1×1 Matrix{Float64}:
 0.4426380973733456
```

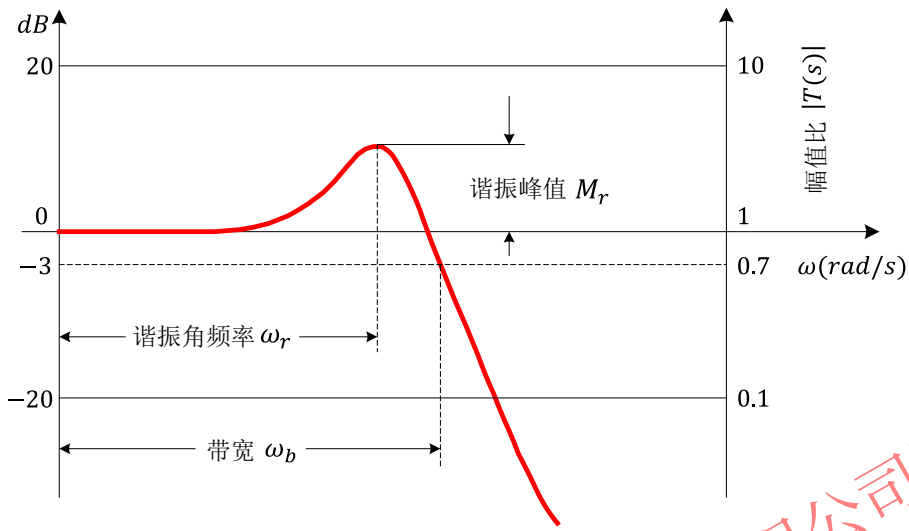


3. 控制系统的频域分析-伯德图

伯德图 – bode()

系统频域指标

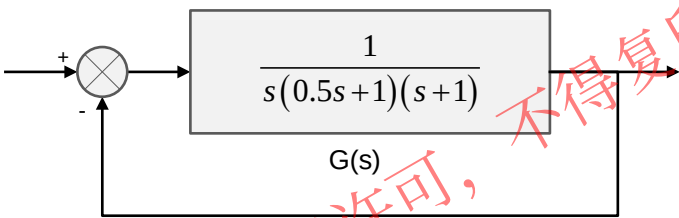
②. 闭环频域指标



闭环频域指标	含义
闭环谐振频率 ω_r	产生谐振峰值对应的频率
闭环谐振峰值 M_r	谐振频率处对应的幅值大小
带宽 ω_b	对数幅频特性的幅值下降到-3dB时对应的频率

通过 `getPeakGain()` 、 `bandwidth()` 函数计算系统闭环谐振峰值与带宽

示例4：考虑如图闭环系统，利用Syslab求取其闭环表达式，并计算其谐振峰值、谐振频率和带宽



```
s = tf('s')
Gopenloop = 1/(s*(0.5*s+1)*(s+1))
Gcloseloop = feedback(Gopenloop)

gpeak, fpeak = getPeakGain(Gcloseloop)
fb = bandwidth(Gcloseloop)
```

输出

```
# 以绝对单位给出谐振峰值，可通过
# 20*log10.(gpeak)计算dB值
julia> gpeak, fpeak =
getPeakGain(Gcloseloop)
(1.8371173070796, 0.8164958474285937)

julia> fb = bandwidth(Gcloseloop)
1x1 Matrix{Float64}:
 1.2619964668070902
```

考虑通过带宽含义计算得到系统带宽

```
# 给定频率向量
w = logspace(-1,1,500)
# 定义系统模型
s = tf('s')
Gopenloop = 1/(s*(0.5*s+1)*(s+1))
Gcloseloop = feedback(Gopenloop)
# 计算频率响应
mag, phase, w = bode(Gcloseloop, w, fig = false)
# 计算系统带宽
n = 1
while 20*log10(mag[n]) >= -3
    global n = n + 1
end
# 获取带宽值
w[n]

julia> w[n]
1.2653317593889428
```



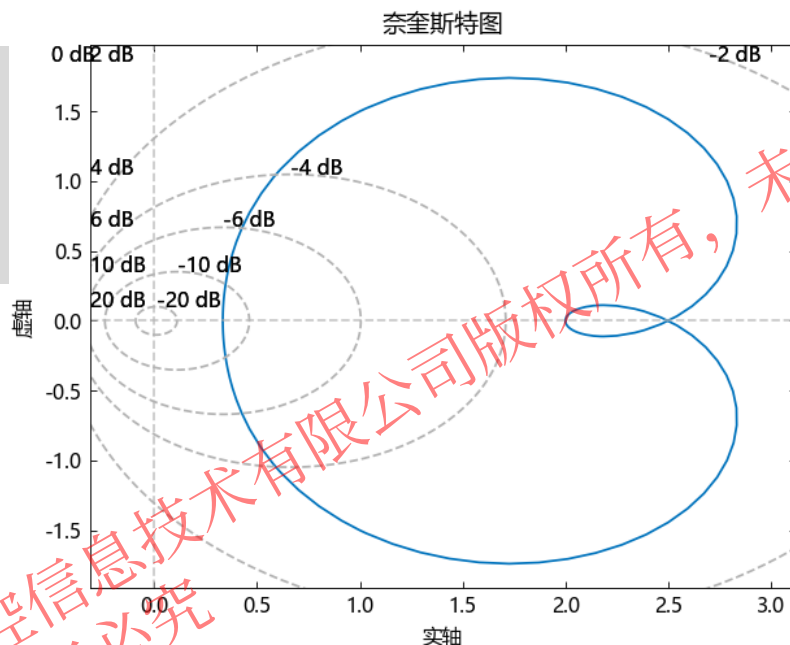
3. 控制系统的频域分析-奈奎斯特图

奈奎斯特图 – nyquist()

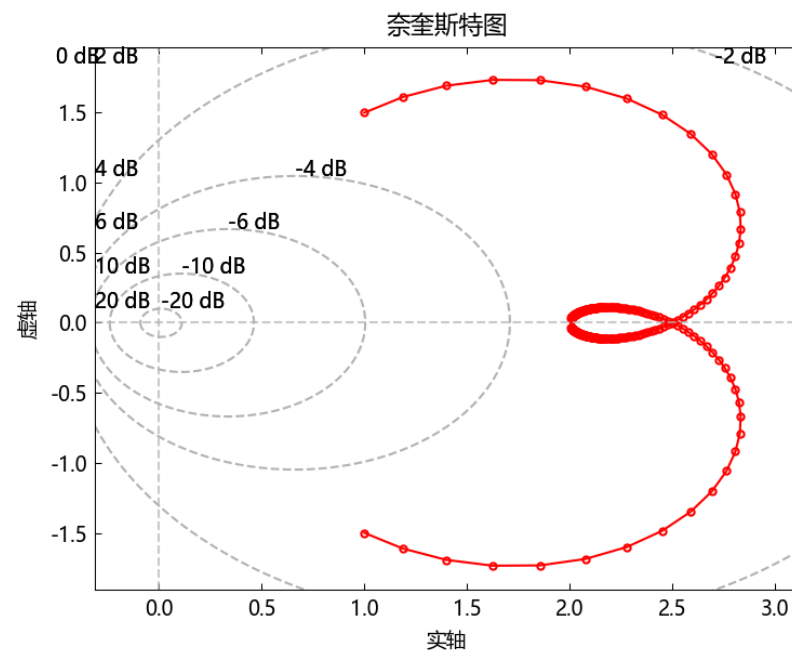
示例5：计算下述系统频率响应并绘制Nyquist图

$$H(s) = \frac{2s^2 + 5s + 1}{s^2 + 2s^2 + 3}$$

```
# 创建传递函数模型
H = tf([2 5 1],[1 2 3])
# 绘制系统Nyquist图
nyquist(H)
# 打开网格线
nyquistgrid(true)
```



```
# 指定频率范围
w = collect(1:0.1:30)
# 绘制系统Nyquist图
nyquist(H, w, "r.-")
# 打开网格线
nyquistgrid(true)
```



Nyquist图的绘制、修饰与bode()用法一致，具体可以参考bode图示例及参考文档，这里不再赘述

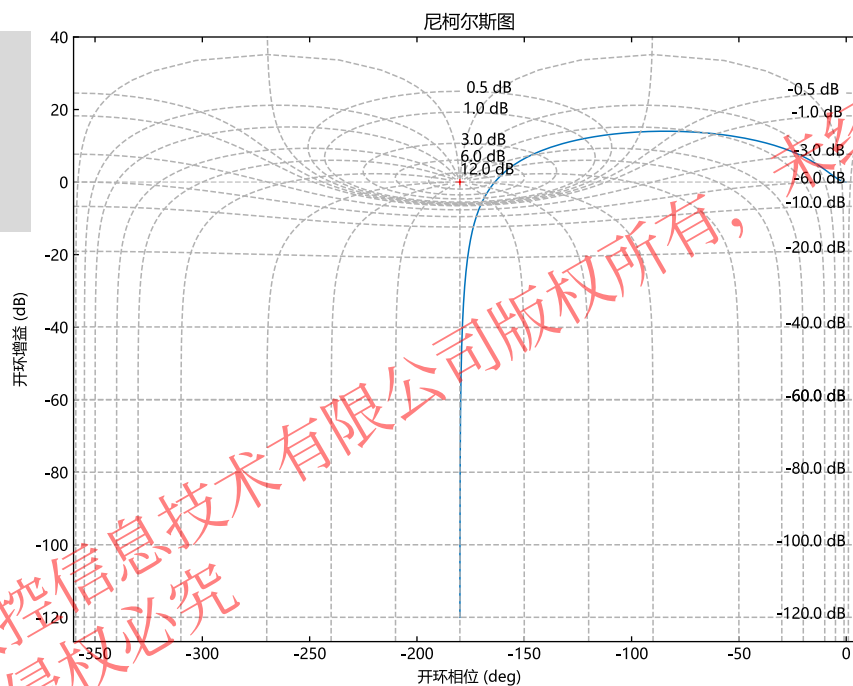
3. 控制系统的频域分析-尼克尔斯图

尼克尔斯图 – nichols()

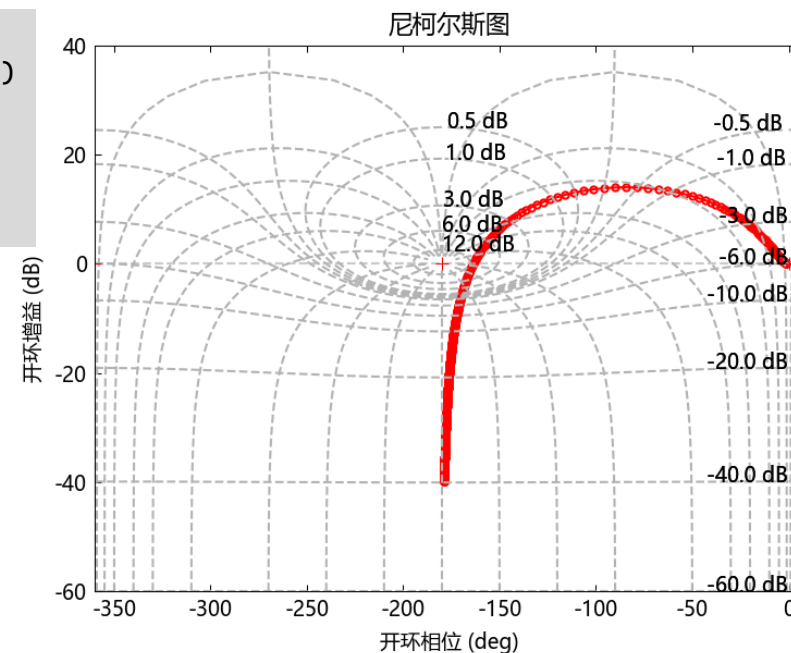
示例6：绘制系统尼克尔斯图

$$G(s) = \frac{100}{s^2 + 2s + 100}$$

```
# 定义系统
G = tf([100],[1 2 100])
# 绘制nichols图并开启网格
nichols(G)
nicholsgrid(true)
```



```
# 指定频率范围
w = collect(1:0.1:100)
# 绘制系统Nyquist图
nichols(G, w, "r--")
# 打开网格线
nicholsgrid(true)
```



Nichols图的绘制、修饰与bode()用法一致，具体可以参考bode图示例及参考文档，这里不再赘述

建立知识规范，营造协同生态
积累工业模型，发展可控平台
融入中国创新，打造先进软件

感谢聆听